



ELEC3609/9609 - Week 8 Lab

07 Security and Testing in Django

In this lab, you will explore fundamental web security issues such as CSRF, SQL injection, and session handling within the Django framework. The lab introduces common pitfalls in insecure web development and how Django provides built-in mechanisms to mitigate these threats.

Individual Task: Individual Task 7 - Understand and Evaluate Django Security

Lab Resources: N/A

Learning Objectives

After completing this lab, students should be able to:

- Identify and explain common web application vulnerabilities including XSS, CSRF, SQL injection, and clickjacking.
- Apply Django's built-in security features such as CSRF middleware, autoescaping, and secure cookie settings.
- Configure secure deployment settings such as ALLOWED_HOSTS, HTTPS enforcement, and secret key management.
- Write effective unit tests for Django models, views, forms, and templates using unittest and Django's test client.

Chapter 1. Common Security Threats and Protections

Web applications are vulnerable to a wide variety of attacks, which can lead to data breaches, loss of user trust, and downtime. In this guide, we will walk through some of the most common security concerns in web development and deployment.

1.1 Cross-Site Scripting (XSS)

XSS is a type of vulnerability where attackers inject malicious scripts into websites. These scripts are executed by users' browsers and can steal sensitive information like cookies, hijack sessions, or perform malicious actions on behalf of the user.

1.1.1 How XSS Attacks Work:

1. The attacker finds a form or input field on your website that does not properly sanitise or escape user input. For example, you have a comment section on your website:

```
<div>{{ user_comment }}</div>
```

2. The attacker submits malicious JavaScript code via this input. For example, if `user_comment` is not properly escaped, an attacker could inject malicious JavaScript like below. This would execute when any user visits the page containing the comment.

```
<script>alert('Your session has been hijacked');</script>
```

1.1.2 Protection Against XSS:

1. **Django's Built-in Protection (Autoescaping)**: Django automatically escapes dangerous characters in HTML templates. This prevents user inputs like `<script>` tags from being executed. By default in Django, every template automatically escapes the output of every variable tag.

```
{% autoescape on %}  
  {{ body }}  
{% endautoescape %}
```

2. **Avoid Using safe Filter in Templates**: Django provides a `safe` filter to mark a string as safe and prevent escaping. However, use this filter cautiously, as it could allow XSS attacks.

```
<p>{{ user_input|safe }}</p> <!-- Avoid this unless absolutely  
necessary -->
```

3. **Input Sanitisation in Backend:** Even though Django's template system escapes content by default, it is a good practice to sanitise input when it is stored or processed, especially for data that will be used in non-HTML contexts like JSON or JavaScript. You can use Python libraries like **Bleach** or **nh3** to sanitise the input before saving it to the database:

```
import bleach
clean_input = bleach.clean(user_input)
```

4. **Setting HTTP-Only Cookie:** This makes it less trivial for an attacker to escalate a cross-site scripting vulnerability into full hijacking of a user's session. Django uses **HTTP-only cookies** for session management by default.

```
# settings.py
SESSION_COOKIE_HTTPONLY = True
```

1.2 Cross-Site Request Forgery (CSRF)

CSRF tricks users into making unwanted actions on a site where they are authenticated. An attacker can craft a malicious request that uses the victim's authentication credentials to perform actions like transferring funds or changing account information.

1.2.1 Common Example of a CSRF Attack

Suppose Alice is logged into her bank `http://bank.com` and has a valid session cookie stored in her browser. Malicious Maria wants to trick Alice into transferring money to Maria's account without Alice's consent.

(With social engineering) Maria can send Alice a malicious link or embeds an image tag like this:

```
<a href="https://bank.com/transfer?acct=Maria&amount=10000">Check out this
cool picture!</a>

<!-- Or as a hidden request -->

```

When Alice clicks the link (or when the image loads), her browser automatically sends the session cookie to `bank.com`, executing the action without Alice's knowledge.

1.2.2 CSRF and HTTP Methods: POST vs. GET

GET Requests and CSRF:

- **GET requests** are typically used to retrieve data and are not meant to modify state (e.g., transfer money, delete an account). However, if an application incorrectly uses GET for sensitive actions, it becomes vulnerable to CSRF even with CSRF token protection.
- **Example:** In the above, the bank uses a GET request to transfer funds, an attacker can exploit this by just phishing a user into clicking a malicious link.
- **Solution:** Avoid using GET requests for any state-changing actions.

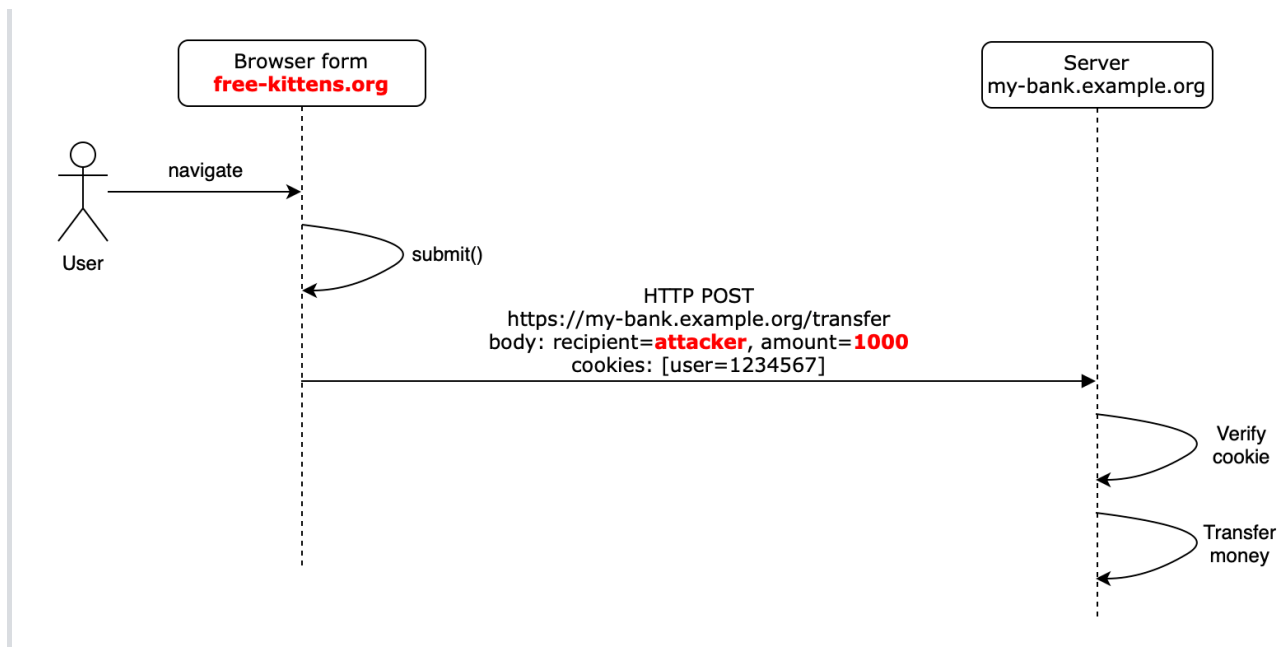
POST Requests and CSRF:

- The only difference between GET and POST attacks is how the attack is being executed by the victim. Suppose the bank now uses POST and the vulnerable request looks like `POST http://bank.com/transfer`.
- **Example:** Such a request cannot be delivered using standard A or IMG tags, but can be delivered using a FORM tags on behalf of Alice.

```
<form action="https://bank.com/transfer" method="POST">
  <input type="hidden" name="account" value="attacker_account">
  <input type="hidden" name="amount" value="1000">
</form>
<script>document.forms[0].submit();</script>
```

- **Solution:** To prevent CSRF attacks on POST requests, websites typically use CSRF tokens as an additional layer of security.

CSRF Attack in POST action



1.2.3 How Does a CSRF Token Prevent CSRF

A CSRF token is a unique, random value generated by the server and associated with the user's session. It is included in every form submission or AJAX request for state-changing actions. When the user submits a request, the token is included alongside the session cookie, and the server checks both to ensure the request is legitimate.

CSRF Token in Django Template

```

<form action="{% url 'transfer' %}" method="POST">
  {% csrf_token %}
  <input type="hidden" name="account" value="recipient_account">
  <input type="hidden" name="amount" value="1000">
  <button type="submit">Transfer</button>
</form>
<script>
  document.forms[0].submit(); // Automatically submits the form
</script>

```

CSRF Token in AJAX Requests:

Now, the server validates the CSRF token before processing the request. If the token is missing or invalid, the server rejects the request. Therefore, a simple phishing email alone will not work.

1.2.4 CSRF Protection in Django

```
fetch('/transfer', {
  method: 'POST',
  headers: {
    'X-CSRFToken': 'random_token_value'
  },
  body: JSON.stringify({ amount: 1000, to: 'Bob' })
});
```

While GET requests are exempt (as they should not modify data), using **HTTPS** and **HSTS** is essential to defend against man-in-the-middle attacks. Disabling CSRF protection should only be done with caution, and cross-subdomain POSTs can be allowed by configuring

`CSRF_TRUSTED_ORIGINS` and `CSRF_COOKIE_DOMAIN`.

Read more about [Cross Site Request Forgery protection in Django](#).

1.2.5 JWT and CSRF in Django REST Framework

When using JWT for authentication, CSRF protection becomes relevant based on how the JWT is stored. If JWT is stored in LocalStorage, it is not automatically sent with requests like cookies, reducing the risk of CSRF. However, XSS becomes a concern. On the other hand, if JWT is stored in cookies as **HTTP-only**, CSRF protection is still required since the browser automatically includes cookies in every request, making CSRF attacks possible. To mitigate this, use CSRF tokens or configure **SameSite attributes** for cookies.

In Django REST Framework, CSRF protection is disabled by default for API views unless you are using session-based authentication. DRF's `APIView` and `ViewSet` classes are typically CSRF exempt. It is important to explicitly re-implement CSRF protection or take alternative measures, depending on your authentication and storage strategy.

For more details, you can check the following:

- <https://github.com/encode/django-rest-framework/issues/6795>
- https://github.com/encode/django-rest-framework/blob/master/rest_framework/views.py#L144
- <https://stackoverflow.com/questions/45945951/jwt-and-csrf-differences>
- <https://stackoverflow.com/questions/49275069/csrf-is-only-checked-when-authenticated-in-drf>
- <https://stackoverflow.com/questions/27067251/where-to-store-jwt-in-browser-how-to-protect-against-csrf>
- <https://mannharleen.github.io/2020-03-19-handling-jwt-securely-part-1/>

1.3 Other Protections

1.3.1 SQL Injection Protection

SQL injection vulnerabilities allow attackers to run arbitrary SQL code on your database, potentially modifying, accessing, or deleting data without proper authorisation. When using Django's models and queriesets (Django ORM), the underlying database driver automatically escapes the SQL, which mitigates the risk of injection. However, when writing raw SQL queries, it's essential to explicitly prevent SQL injection by safely parameterising inputs.

1.3.2 Clickjacking Protection

Clickjacking is an attack where a malicious user hijacks clicks meant for a top-level site, routing them to a hidden, malicious page underneath. Attackers may use this method to display legitimate content, such as a bank's login page, while capturing user credentials in an invisible iframe they control. Django combats this attack through the [X-Frame-Options middleware](#), which prevents your site from being rendered inside an iframe in compatible browsers. Read more about [Clickjacking Protection in Django](#).

1.3.3 Enforcing TLS/HTTPS

TLS/HTTPS encrypts communication between the server and client, ensuring sensitive data, including authentication credentials, is not sent in plain text. Enabling HTTPS is crucial for site security. Django offers additional protections when HTTPS is enabled:

- [SECURE_PROXY_SSL_HEADER](#): Ensures security even when traffic passes through a non-HTTPS proxy.
- [SECURE_SSL_REDIRECT](#): Automatically redirects all HTTP requests to HTTPS.
- [HTTP Strict Transport Security \(HSTS\)](#): This HTTP header enforces the use of HTTPS for all future connections to the site. Combined with redirecting HTTP traffic to HTTPS, it ensures that HTTPS is always used once a secure connection is established. You can configure HSTS using `SECURE_HSTS_SECONDS` and `SECURE_HSTS_INCLUDE_SUBDOMAINS`.
- [Secure Cookies](#): By setting `SESSION_COOKIE_SECURE` and `CSRF_COOKIE_SECURE` to `True`, cookies are sent only over HTTPS, further safeguarding sensitive information.

1.4 Security Settings for Deployment

- **ALLOWED_HOSTS**: Restrict which domains can serve your Django app to prevent HTTP Host header attacks.

```
ALLOWED_HOSTS = [ 'yourdomain.com', 'www.yourdomain.com' ]
```

- **SECRET_KEY in .env**: Keep your secret keys and sensitive data out of your codebase by using environment variables stored in a .env file.

```
SECRET_KEY = os.getenv( 'SECRET_KEY' )
```

- **DEBUG Mode**: Never run your app with `DEBUG=True` in production, as it exposes sensitive information.

```
DEBUG = False
```

- **CORS** and **CSRF** Trusted Origins: Setup CORS and CSRF Trusted Origins to control which domains can interact with your API securely.

```
CORS_ALLOWED_ORIGINS = [ 'https://yourfrontend.com' ]  
CSRF_TRUSTED_ORIGINS = [ 'https://yourfrontend.com' ]
```

- **Database Permissions**: Ensure your database user (that connects with Django application) has limited permissions (avoid using root). Limit CREATE, ALTER, or DROP permissions to reduce the impact of SQL injections.
- **Linux File Permission Management**: Protect sensitive files by setting appropriate permissions on your linux server. For example:

```
chmod 600 /path/to/.env
```

Chapter 2. Django Testing

Django provides a robust suite of built-in testing tools that allow developers to write tests to ensure their application works as expected. Testing is critical to verify the functionality of your models, views, forms, databases and even your templates. You should write tests for anything that is crucial to your application, particularly features that handle user data, interact with database, or display dynamic content. Django's testing framework integrates seamlessly with Python's `unittest`, making it easy to create and manage tests.

Read more about [Testing in Django](#).

2.1 Testing Structure and How to Run Tests

Django organises tests using classes derived from `TestCase`, and individual test methods are prefixed with `test_`. All test files should be placed inside an app's `tests.py` or a dedicated `tests` directory, like:

```
til_app/  
    tests.py  
    /tests/  
        __init__.py  
        test_models.py  
        test_forms.py  
        test_views.py
```

To run your tests, use the command:

```
python manage.py test
```

This command automatically discovers and runs all tests across your project, and provides feedback on which tests pass or fail. The test runner provides many testing options, see the Django [test runner](#) for more information.

2.2 What to Test?

2.2.1 Testing Models

Testing models ensures the database structure and logic behave correctly. This often includes verifying field default values, model methods, and database relationships.

```
from django.test import TestCase  
from .models import Post  
  
class PostModelTests(TestCase):  
    def test_default_visibility_should_be_public(self):  
        post = Post(subject="Test Post", content="This is a test post.")
```

```
self.assertEqual(post.public, True)
```

The `assertTrue`, `assertFalse`, `assertEqual` are standard assertions provided by **unittest**. There are other standard assertions in the framework, and also **Django-specific assertions** to test if a view redirects (`assertRedirects`), to test if a particular template has been used (`assertTemplateUsed`), etc.

2.2.2 Testing Forms

Forms are crucial for user input validation. You should test that forms correctly validate input, handle missing fields, and reject invalid submissions.

```
from django.test import TestCase
from .forms import PostForm

class PostFormTests(TestCase):
    def test_post_form_valid(self):
        form_data = {'subject': 'Test Post', 'content': 'Test content'}
        form = PostForm(data=form_data)
        self.assertTrue(form.is_valid())
```

2.2.3 Testing Views

Django views are tested to verify the correct response, template rendering, and context variables. You can simulate HTTP requests using **Django's test client** to check the behaviour of views.

```
from django.test import TestCase
from django.urls import reverse

class PostViewTests(TestCase):
    def test_post_list_view(self):
        response = self.client.get(reverse('post_list'))
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Post List")
```

Check the [link](#) for more information on setting up the request client, how it functions and how to structure the test cases. **The test client is not supposed to replace frontend testing**, as we will still need something to verify that the JavaScript is functioning as expected and any other layout or styling issues.

2.2.4 Testing Templates

Templates should be tested for correct rendering, dynamic content, and layout. Django's test client allows you to verify if the correct template is rendered with the expected content.

```
from django.test import TestCase
from django.urls import reverse

class TemplateTests(TestCase):
    def test_home_template(self):
        response = self.client.get(reverse('home'))
        self.assertTemplateUsed(response, 'home.html')
```

2.2.5 Using the Test Database

Use **Django's test database** to test model behaviour without affecting production data. You can use **fixtures** or `setUpTestData()` to create data shared across tests.

```
from django.test import TestCase
from .models import Post

class PostDatabaseTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.post = Post.objects.create(subject="Test Post", content="Test content")

    def test_post_creation(self):
        self.assertEqual(Post.objects.count(), 1)
```

Usually constructing objects directly in `setUpTestData()` is sufficient for testing the database.

2.3 Code Coverage in Django

Code coverage helps identify parts of your code that are not covered by tests. Django integrates with the **Coverage.py** tool to measure how much of your codebase is tested. To use it:

1. Install **Coverage.py**:

```
pip install coverage
```

```
coverage run manage.py test
```

```
coverage report # or coverage html for a more visual report
```

Check the [link](#) for details on how to integrate with **Coverage.py**.

2.4 Frontend Testing Tools

While Django's built-in testing tools focus on the backend, frontend testing is essential for validating JavaScript, user interactions, and layout consistency. Here are some tools you can use:

- **Selenium**: Automates browser interactions, simulating real user actions like clicks, form submissions, and navigation across multiple pages. It is ideal for testing complete user flows in a browser.
- **Jest**: A JavaScript testing framework designed for testing frontend logic. It provides easy-to-use syntax for writing unit tests.
- **Jasmine**: A behaviour-driven testing framework for JavaScript. It is often used for writing unit and integration tests for JavaScript code, focusing on simple syntax and intuitive test writing.

Further Readings

In addition to the links provided in the content, there are some excellent reading materials available for a deeper understanding of security and testing.

Common Security Topics

- [Security in Django](#)
- [Cross Site Request Forgery - OWASP](#)
- [Cross-Site Request Forgery Prevention Cheat Sheet - OWASP](#)
- [Django web application security - Mozilla](#)
- [Cross-Origin Resource Sharing - Mozilla](#)

- [Web Security Cheat Sheet - Mozilla](#)

Testing in Django

- [Testing a Django web application - Mozilla](#)
- [Writing and running tests - Django](#)
- [Testing tools - Django](#)
- [Advanced testing topics - Django](#)

Credit

This lab content was authored and maintained by **Jiawen Wen**, with materials adapted from prior offerings and updated to align with 2026 delivery. All standards used in this lab (IEEE, ISO/IEC) are referenced for educational purposes under fair use and are available to enrolled students via the university's licensed repository.